

Prática 1 – BLE Radar (Scan + Proximidade + Radar)

Nome: Edison Andres Arteaga Lopez

1) Objetivos da prática

Construir um app que executa no Android do Raspberry Pi 5, faz scan BLE, mantém uma lista atualizada de dispositivos, mostra RSSI bruto e RSSI suavizado (média móvel), classifica proximidade (Muito perto / Perto / Médio / Longe) e renderiza um radar (pontos mais perto do centro para sinal mais forte).

2) Componentes da API BLE usados nesta prática

A prática utiliza a pilha BLE nativa do Android para realizar varredura (scan) de dispositivos próximos e coletar o RSSI (intensidade de sinal) associado a cada anúncio recebido. Abaixo estão os principais componentes usados, com objetivo e papel no fluxo.

Fluxo geral (visão de arquitetura):

BluetoothManager → BluetoothAdapter → BluetoothLeScanner → startScan(ScanSettings) → ScanCallback(onScanResult → ScanResult → BluetoothDevice + RSSI)

A) BluetoothManager

O BluetoothManager é o ponto de entrada oficial para acessar o subsistema Bluetooth no Android via `getSystemService(Context.BLUETOOTH_SERVICE)`. Ele fornece o BluetoothAdapter, que representa o adaptador Bluetooth do dispositivo.

B) BluetoothAdapter

O BluetoothAdapter representa o adaptador Bluetooth do dispositivo (hardware/stack). Ele é usado para verificar se o Bluetooth está habilitado (`isEnabled()`) e para obter o scanner BLE (`bluetoothLeScanner`). Na prática, o BluetoothAdapter funciona como “guardião” do fluxo: se o Bluetooth estiver desligado, o app não tenta iniciar scan e informa o usuário, evitando falhas.

C) BluetoothLeScanner

O BluetoothLeScanner é a API responsável por iniciar e parar a varredura BLE. Ele expõe `startScan()` e `stopScan()` e entrega resultados de maneira assíncrona via ScanCallback. Este é o componente central da prática, pois é ele que efetivamente executa o scan e produz eventos com dados de dispositivos detectados e RSSI.

D) ScanSettings

`ScanSettings` define parâmetros do scan, especialmente o modo de varredura. Nesta prática usamos `SCAN_MODE_LOW_LATENCY` para obter alta responsividade, permitindo observar variações de RSSI quando o aluno aproxima/afasta o celular do Raspberry. O trade-off é maior consumo de energia; em aplicações reais pode-se preferir modos mais econômicos dependendo do caso de uso.

5) ScanCallback

`ScanCallback` é a interface de retorno usada pelo Android para notificar o app sobre resultados do scan. É um elemento didático importante porque evidencia o modelo assíncrono: o app não “puxa” resultados; ele recebe eventos do sistema. Nesta prática, no `onScanResult()`, atualizamos uma estrutura (mapa por endereço) e recalculamos RSSI suavizado e proximidade.

6) ScanResult

Cada evento do `ScanCallback` traz um `ScanResult`, que encapsula o `BluetoothDevice` detectado e métricas do momento (por exemplo, RSSI). O RSSI é o dado-chave para o “radar” e para a classificação de proximidade, mas é naturalmente ruidoso e sensível ao ambiente (obstáculos, reflexões, orientação do aparelho).

7) BluetoothDevice

O `BluetoothDevice` representa o dispositivo remoto detectado durante o scan. Nesta prática, usamos principalmente:

- `address`: identificador estável para indexar e atualizar o mesmo dispositivo ao longo do tempo;
- `name`: nome do dispositivo quando disponível (pode ser null, dependendo do anúncio BLE e do dispositivo remoto).

É importante ressaltar que “ver um dispositivo no scan” não significa estar conectado a ele; é apenas a detecção de anúncios BLE.

3) Permissões e compatibilidade

A partir do Android 12 (API 31), o modelo de permissões para Bluetooth foi aprimorado. Para scan BLE, o app deve solicitar permissões explícitas em runtime:

- `BLUETOOTH_SCAN`: autoriza varredura BLE;
- `BLUETOOTH_CONNECT`: autoriza operações que acessam/gerenciam conexões e certas

informações do dispositivo.

No Manifest, usamos `usesPermissionFlags="neverForLocation"` para declarar que o scan não é usado para fins de localização.

2

4) Observação: RSSI ≠ distância em metros

O RSSI é uma aproximação de intensidade de sinal e varia com ambiente, obstáculos, orientação do dispositivo e interferências. Nesta prática, o “radar” e a “proximidade” são heurísticas para visualização e comparação relativa. A prática usa suavização (EMA) para reduzir ruído e tornar a experiência em sala mais consistente.

5) Arquivos

Todos os arquivos da prática estão no Google Classroom. Alguns arquivos você deve substituir todo o seu conteúdo. Em outros arquivos você deve apenas adicionar algumas linhas e manter o que foi criado pelo Android Studio. Cuidado com os nomes dos arquivos e packages.

Parte A: Estrutura do projeto

A estrutura de pastas do projeto deve estar parecida com essa:

```
Shell
app/src/main/
    AndroidManifest.xml
    java/com/example/bleradar/
        MainActivity.kt
    model/
        DeviceInfo.kt
    util/
        SignalUtils.kt
    ui/
        DeviceAdapter.kt
        RadarView.kt
    res/
        layout/
            activity_main.xml
            item_device.xml
```

Para criar um projeto no Android Studio, execute os seguintes passos:

1. File → New → New Project
2. Template: Empty Views Activity
3. Name: Bleradar

4. Package name: `com.example.bleradar`
5. Language: Kotlin
6. Minimum SDK: 26
7. Finish

3

Parte B - Códigos

Observação: As vezes pode ser mais interessante mudar a visão para “Project” no Android Studio. A visão padrão é “Android”.

1) Ativar ViewBinding (build.gradle.kts)

Acesse o arquivo `app/build.gradle.kts`. Dentro de `android { ... }` adicione o `buildFeatures`:

```
Kotlin
android {
    .
    .
    .
    buildFeatures {
        viewBinding = true
    }
}
```

Clique em Sync Now.

2) AndroidManifest (BLE + permissões)

Adicione o seguinte código no `app/src/AndroidManifest.xml`:

```
XML
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!--
    FEATURE BLE:
    - Diz ao sistema que o app usa Bluetooth Low Energy.
    - android:required="true" impede instalação em devices sem BLE. Apenas para
    didática.
    -->
```

```

<uses-feature
android:name="android.hardware.bluetooth_le"
android:required="true" />

```

4

```

<!--
PERMISSÕES ANDROID 12+ (API 31+):
- A partir do Android 12, Bluetooth tem permissões "próprias". -
BLUETOOTH_SCAN: permite fazer varredura (scan).
- BLUETOOTH_CONNECT: permite acessar informações/conectar. -
usesPermissionFlags="neverForLocation": declara que o scan não é para
localização.
-->
<uses-permission
android:name="android.permission.BLUETOOTH_SCAN"
android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />

<application
    ...
</application>
</manifest>

```

3) UI (XML)

3.1) activity_main.xml

Substitua o arquivo `app/src/main/res/layout/activity_main.xml` pelo código xml disponibilizado no Google Classroom da disciplina (Prática 1).

Observação: O Android Studio vai apresentar um erro, pois `com.example.bleradar.ui.RadarView` ainda não existe. Ignorar.

3.2 item_device.xml

Crie o arquivo `item_device.xml` dentro da pasta `layout`. Clique no botão direito em `layout` → `New` → `File`. Digite o nome do arquivo e pressione enter. O arquivo vai estar na localização `app/src/main/res/layout/item_device.xml`.

Copie e cole o conteúdo do arquivo disponibilizado.

4) Criar packages e arquivos .kt

Vá em `app > main > java > com.example.bleradar`.

- Clique direito em com.example.bleradar → New → Package → model
- Clique direito em com.example.bleradar → New → Package → util

5

- Clique direito em com.example.bleradar → New → Package → ui

Após, crie os arquivos necessários em cada projeto. Clique direito no package (ex.: ui) → New → Kotlin Class/File → File → nome do arquivo.

Crie exatamente estes arquivos:

- model/DeviceInfo.kt
- util/SignalUtils.kt
- ui/DeviceAdapter.kt
- ui/RadarView.kt

5) Substitua os arquivos .tk criados

Após criar os arquivos, substitua o seu conteúdo pelo conteúdo dos arquivos disponibilizados no Google Classroom.

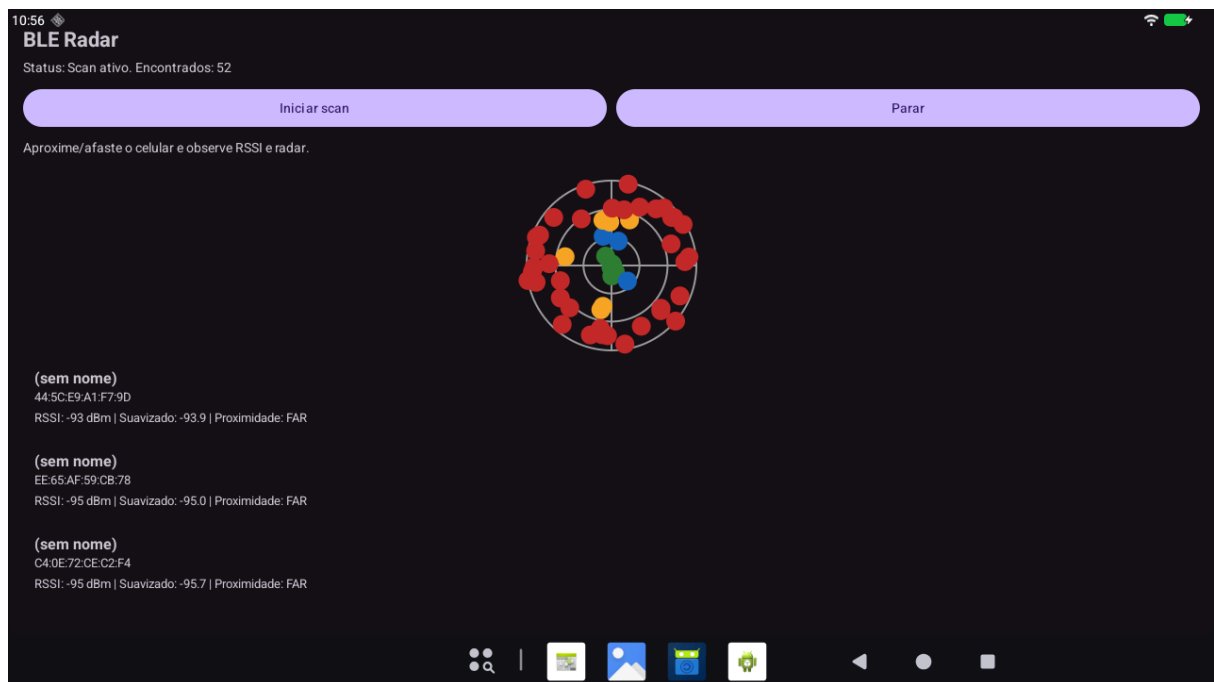
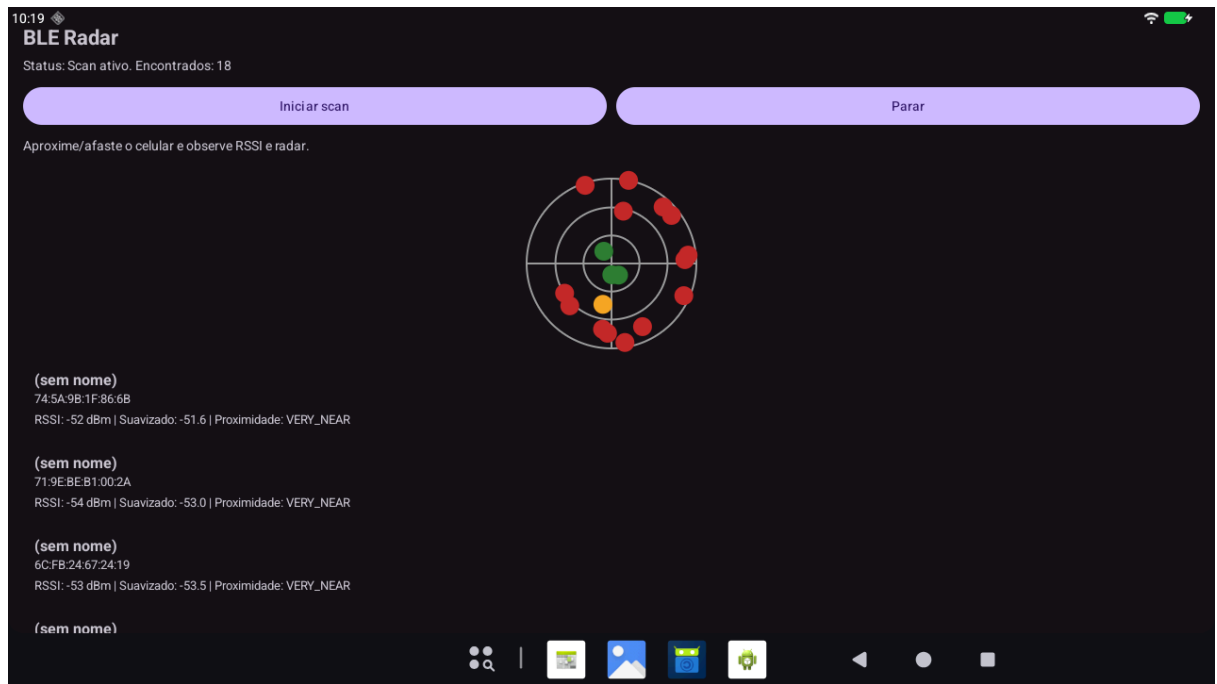
9) MainActivity.kt

Substitua o conteúdo do arquivo MainActivity.kt pelo arquivo disponibilizado no Google Classroom.

Clique em Sync Now. Se aparecer algum erro de permissão do bluetooth, espere para verificar se ele desaparece.

10) Como compilar e rodar no Raspberry

- adb devices deve listar o Raspberry como device
- No Android Studio selecione o Raspberry como target
- ▶ Run
- No Raspberry: ligar Bluetooth
- No app: Iniciar scan
- Aproximar/afastar celular e observar RSSI + radar



Observações

Se precisar remover o aplicativo do Raspberry, digite no terminal:

Shell

```
adb uninstall com.example.bleradar
```

Parte C - O que deve ser entregue

Questão 1) Tente descobrir quais são os dispositivos encontrados no BLE Radar. Faça considerações sobre o valor do RSSI e a classificação de distância dos dispositivos. Quais são os impactos dos obstáculos entre os dispositivos. A classificação de distância está próxima da realidade.

Na imagem, todos os dispositivos listados aparecem como “(sem nome)”. Isso ocorre porque o anúncio BLE não inclui o nome local completo (campo Complete Local Name). Mesmo assim, o endereço MAC (ex.: 74:5A:9B:1F:86:6B) permite identificar unicamente cada dispositivo. Possíveis fontes desses sinais:

Smartphones com Bluetooth ligado (em modo “detectável”)

Fones de ouvido, smartwatches ou outros wearables

Valores de RSSI e classificação

RSSI bruto: variando entre -52 e -54 dBm (forte, típico de poucos centímetros até ~0,5 m).

RSSI suavizado (média exponencial): valores como -51,6 e -53,0 – mais estáveis.

Proximidade: VERY_NEAR – indica que o dispositivo está muito próximo ao Raspberry Pi.

Impacto de obstáculos

Qualquer objeto entre o Raspberry e o dispositivo emissor reduz o RSSI:

Uma folha de papel: atenuação pequena (1–3 dB).

Mão humana: atenuação de 10–20 dB, alterando a classificação para NEAR ou MID.

Parede de concreto: pode derrubar o sinal para -80 dBm ou menos (FAR).

A classificação está próxima da realidade?

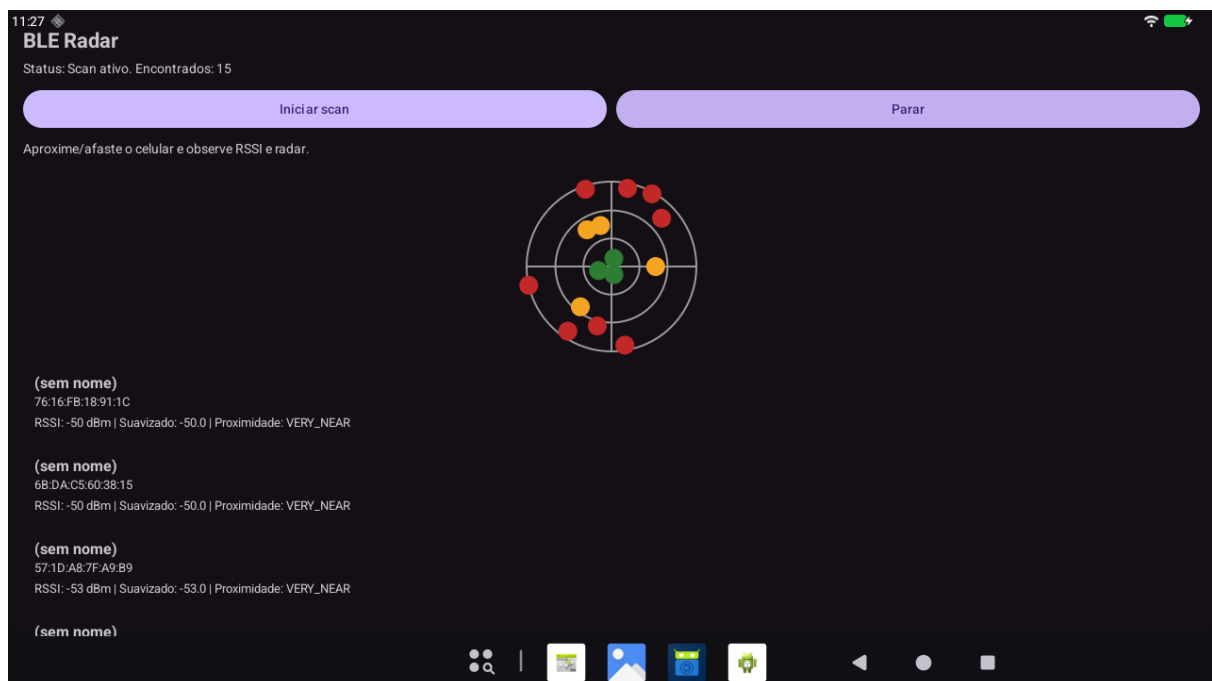
Sim, neste caso. Com RSSI acima de -50 dBm, a classificação VERY_NEAR é coerente com a distância real (menos de 0,5 m). Entretanto, se houvesse obstáculos ou reflexões, o mesmo RSSI poderia corresponder a uma distância maior. Por isso, a prática enfatiza que é uma heurística relativa, não uma medição absoluta de distância.

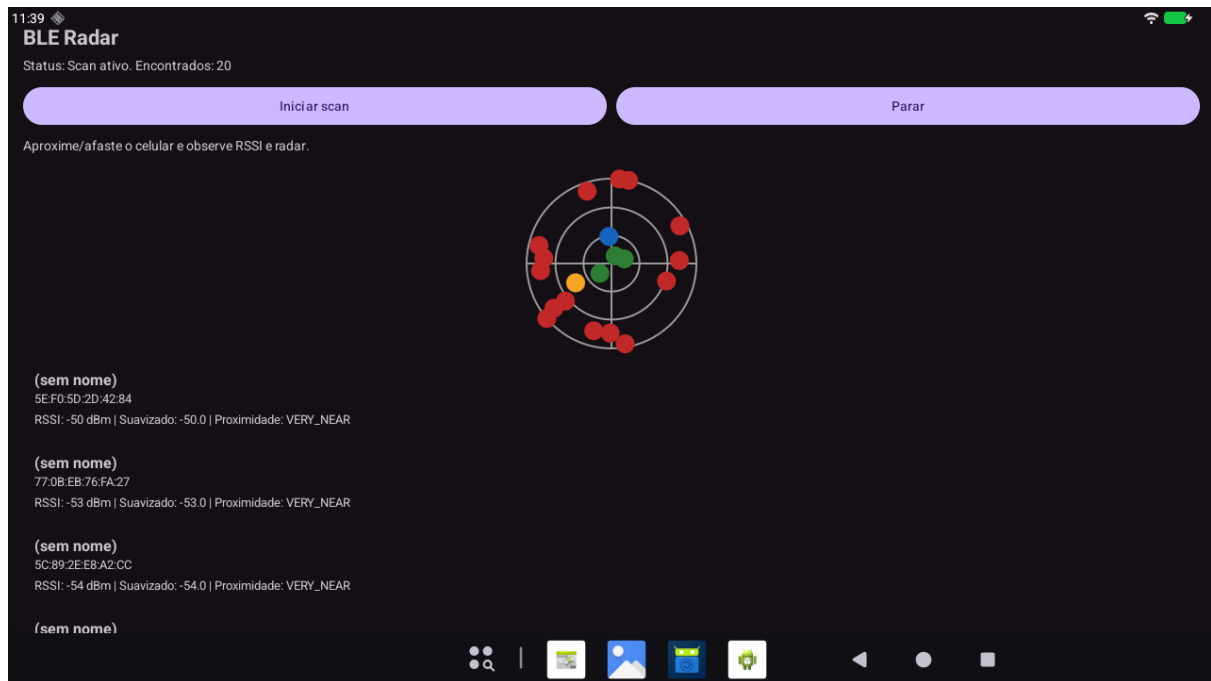
Questão 2) Encontre a função que calcula a média móvel do RSSI. Faça alterações e verifique o resultado. Por exemplo, retorne o valor bruto do RSSI sem a média móvel. Como o RSSI se comporta.

No [MainActivity.kt](#), mude “// existing.rssiSmoothed = SignalUtils.ema(existing.rssiSmoothed, rssi)”

`existing.rssiSmoothed = rssi.toDouble() // usa o RSSI cru”`

Altere a função para ignorar o histórico e retornar apenas o RSSI atual:





Comportamento observado após a alteração

O valor de `rssSmoothed` (agora bruto) oscilará fortemente a cada pacote recebido – variações de ± 5 a ± 15 dBm são comuns.

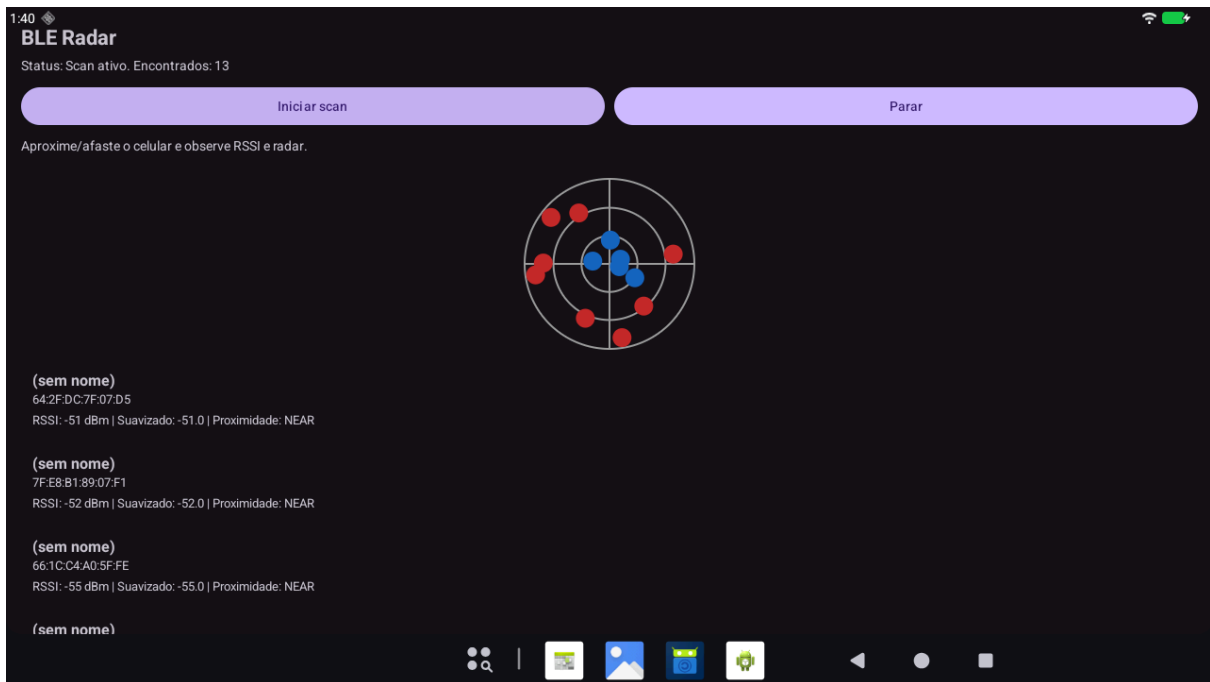
A classificação de proximidade (`SignalUtils.proximityFromRssi`) será recalculada a cada oscilação, podendo pular de `VERY_NEAR` para `NEAR` e voltar em segundos, mesmo com o celular parado.

O radar (que usa `rssSmoothed` para calcular `radarRadiusNorm`) desenhará pontos que saltam de posição repetidamente, tornando a visualização confusa.

A suavização EMA é essencial para este tipo de aplicação porque reduz o ruído de curto prazo e permite que o usuário perceba a tendência real do sinal (aproximando ou afastando). Teste essa alteração e depois restaure o código original para confirmar a diferença.

Questão 3) Faça uma calibragem das proximidades `VERY_NEAR`, `NEAR`, `MID` e `FAR`. Atualize a função `proximityFromRssi`.

```
2 Usages
fun proximityFromRssi(rssiSmoothed: Double): Proximity {
    return when {
        //rssiSmoothed >= -55 -> Proximity.VERY_NEAR
        //rssiSmoothed >= -65 -> Proximity.NEAR
        //rssiSmoothed >= -75 -> Proximity.MID
        rssiSmoothed >= -45 -> Proximity.VERY_NEAR
        rssiSmoothed >= -63 -> Proximity.NEAR
        rssiSmoothed >= -76 -> Proximity.MID
        else -> Proximity.FAR
    }
}
```



Além das questões acima, o código fonte do projeto também deve ser entregue no Classroom da disciplina.

